

Tribhuvan University
IOST
Prithvi Narayan Campus



Lab Report of Net Centric Computing by:

Vivek Paudel

Sixth Semester

Section: A

Roll no: 80012185 (Class: 24)

Submitted To:

Thakur Prasad Upadhyaya

(NCC Teacher)

INDEX

Name: Vivek Paudel

Roll No: 80012185 (24)

SN	Name of Experiment	Date of submission	Signature
1	Working with Object-Oriented Programming (OOP) Concepts, Exception Handling, and LINQ in C#		
2	ASP.NET Core Web App with Razor Pages		
3	Explore MVC Web App in ASP.NET Core		
4	Explore MVC Web API in ASP.NET Core		

Lab 1: Working with Object-Oriented Programming (OOP) Concepts, Exception Handling, and LINQ in C#

2. Theory

2.1 Object-Oriented Programming (OOP) and .NET Architecture

Object-Oriented Programming (OOP) serves as the foundational design paradigm within the .NET Common Type System (CTS) and Common Language Runtime (CLR), allowing developers to organize complex applications into modular, reusable compiled assemblies. Core principles such as inheritance establish a clear "is-a" hierarchical relationship where a derived class inherits and specializes properties from a base class, whereas aggregation implements a "has-a" association by embedding generic collection types like `List<T>` to manage related entities. Furthermore, encapsulation is strictly enforced through access modifiers (`public`, `private`, `protected`, `internal`) to control visibility across assemblies, while polymorphism enables flexible method behavior through compile-time overloading, dynamic run-time overriding via virtual and override v-tables, and compile-time method hiding using the `new` keyword.

2.2 Robust Exception Handling Mechanics

Exception handling provides a robust, fault-tolerant mechanism to manage runtime anomalies gracefully, ensuring that unexpected errors do not trigger catastrophic application crashes. In C#, this workflow relies on structured blocks where operations prone to failure—such as parsing user inputs, file I/O, or arithmetic calculations—are wrapped inside a `try` block for runtime monitoring. When an exceptional condition arises, execution immediately jumps to a matching `catch` block designed to intercept specific error types (such as `FormatException` or `DivideByZeroException`), enabling targeted recovery or logging. Finally, an optional `finally` block ensures that crucial cleanup operations or resource releases execute unconditionally, maintaining system stability regardless of whether an exception occurred.

2.3 Language Integrated Query (LINQ) and Lambda Expressions

Language Integrated Query (LINQ) bridges the gap between object-oriented logic and declarative data manipulation by embedding query capabilities natively into the C# language syntax. It allows developers to query collections, arrays, and databases using a readable, SQL-like query syntax (`from`, `where`, `orderby`, `select`) that is translated by the compiler into efficient executable logic. Alongside query expressions, C# natively supports lambda expressions using the lambda operator (`=>`), offering a concise functional programming style for method-based querying (such as `.Where()` and `.OrderBy()`), which together streamline data filtering, sorting, and transformation with minimal boilerplate.

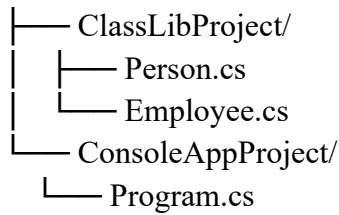
2.4 Console Application Execution and Architecture

The console application component acts as the executable entry point (`Program.cs`) of the solution, bridging user interaction with the encapsulated logic defined in the external class library. It handles runtime initialization by instantiating domain models, populating generic lists for aggregated data, and executing iterative traversal via standard `for` and `foreach` loops. Operating through a text-based interface, a console application handles inputs, reads user commands, and outputs formatted results directly to the command line, orchestrating the execution flow for testing exception-handling safeguards and evaluating datasets using both LINQ query syntax and lambda expressions.

3. Demonstration

3.1 File Structure

Labs.sln



3.2 File Description and Source Code

i. The Base Class ('Person.cs')

Acts as the base class encapsulating fundamental member properties ('FullName' and 'Age') alongside a 'virtual PrintDetails' method to support polymorphism.

```
namespace ClassLibProject
{
    public class Person
    {
        public string FullName { get; set; }
        public int Age { get; set; }

        // Virtual base method for runtime polymorphism
        public virtual void PrintDetails()
        {
            System.Console.WriteLine($"Full Name: {FullName} | Age: {Age}");
        }
    }
}
```

ii. The Derived Class with Aggregation (Employee.cs)

Highlights inherited attributes, specialized properties (Code, BaseSalary), generic collections (List<Employee> Subordinates), method overriding, and method overloading.

```
using System.Collections.Generic;
```

```
namespace ClassLibProject
{
    public class Employee : Person
    {
        public string Code { get; set; }
        public double BaseSalary { get; set; }

        // Generic list field for aggregation ("has-a" relationship)
        public List<Employee> Subordinates { get; set; } = new List<Employee>();
    }
}
```

```

// Overriding method (Runtime Polymorphism)
public override void PrintDetails()
{
    base.PrintDetails();
    System.Console.WriteLine($"Employee Code: {Code} | Base Salary: NPR
{BaseSalary:N2}");
}

// Method Overloading (Compile-time Polymorphism)
public void CalculatePay(double allowance)
{
    BaseSalary += allowance;
}

public void CalculatePay(double allowance, double taxAmount)
{
    BaseSalary += (allowance - taxAmount);
}
}
}

```

iii. The Entry Point and Logic (Program.cs)

Demonstrates object instantiation tailored for **Vivek Paudel**, for and foreach loop traversals, try-catch-finally exception handling blocks, and LINQ/lambda queries over Nepalese cities.

```

using System;
using System.Collections.Generic;
using System.Linq;
using ClassLibProject;

namespace ConsoleAppProject
{
    class Program
    {
        static void Main(string[] args)
        {
            // Instantiating main lead object for Vivek Paudel
            Employee leadManager = new Employee
            {
                FullName = "Vivek Paudel",
                Age = 23,
                Code = "EMP-701",
                BaseSalary = 105000.00
            };

            // Aggregating team members with updated names and details
            leadManager.Subordinates.Add(new Employee { FullName = "Rohan KC", Age = 27, Code
= "EMP-702", BaseSalary = 54000 });

```

```

        leadManager.Subordinates.Add(new Employee { FullName = "Bhawana Adhikari", Age = 25, Code = "EMP-703", BaseSalary = 51000 });
        leadManager.Subordinates.Add(new Employee { FullName = "Sajan Bhattarai", Age = 29, Code = "EMP-704", BaseSalary = 62000 });
        leadManager.Subordinates.Add(new Employee { FullName = "Prashant Thapa", Age = 26, Code = "EMP-705", BaseSalary = 48000 });
        leadManager.Subordinates.Add(new Employee { FullName = "Kritika Neupane", Age = 28, Code = "EMP-706", BaseSalary = 59000 });

```

```

        Console.WriteLine("=====");
        Console.WriteLine("    LEAD MANAGER INFORMATION    ");
        Console.WriteLine("=====");
        leadManager.PrintDetails();

        // Traversing aggregated collection via 'for' loop
        Console.WriteLine("\n--- Subordinates List (Iterated via 'for' loop) ---");
        for (int index = 0; index < leadManager.Subordinates.Count; index++)
        {
            Console.WriteLine($"[{index + 1}] Name: {leadManager.Subordinates[index].FullName} | Code: {leadManager.Subordinates[index].Code}");
        }

        // Traversing aggregated collection via 'foreach' loop
        Console.WriteLine("\n--- Subordinates Pay Scale (Iterated via 'foreach' loop) ---");
        foreach (var staff in leadManager.Subordinates)
        {
            Console.WriteLine($"Staff: {staff.FullName,-20} | Base Salary: NPR {staff.BaseSalary:N2}");
        }

        // Execute Exception Handling Module
        ExecuteExceptionHandling();

        // Execute LINQ & Lambda Query Module
        ExecuteLinqAndLambdaQueries();
    }

```

```

static void ExecuteExceptionHandling()
{
    Console.WriteLine("\n=====");
    Console.WriteLine("    EXCEPTION HANDLING TEST BLOCK    ");
    Console.WriteLine("=====");
    try
    {
        Console.Write("Enter a number to divide 5000 by: ");
        string rawInput = Console.ReadLine();
        int divisorValue = int.Parse(rawInput);
    }
    catch
    {
    }
}

```

```

        int outputResult = 5000 / divisorValue;
        Console.WriteLine($"Computation Output: 5000 / {divisorValue} = {outputResult}");
    }
    catch (FormatException)
    {
        Console.WriteLine("Error Caught: Input text could not be converted to a valid integer.");
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine("Error Caught: Arithmetic division by zero is mathematically
undefined.");
    }
    finally
    {
        Console.WriteLine("Status: Exception block processing finalized cleanly.");
    }
}

static void ExecuteLinqAndLambdaQueries()
{
    // Array of Nepalese cities
    string[] citiesInNepal = {
        "Kathmandu", "Pokhara", "Lalitpur", "Bharatpur",
        "Biratnagar", "Birgunj", "Dharan", "Butwal",
        "Hetauda", "Nepalgunj", "Damauli", "Bhimad", "Khairani"
    };

    Console.WriteLine("\n=====");
    Console.WriteLine("    LINQ & LAMBDA QUERY RESULTS    ");
    Console.WriteLine("=====");

    // LINQ Query: Filter cities starting with 'B' and order alphabetically
    var queryCitiesStartingWithB = from cityItem in citiesInNepal
                                    where cityItem.StartsWith("B", StringComparison.OrdinalIgnoreCase)
                                    orderby cityItem ascending
                                    select cityItem;

    Console.WriteLine("\n[LINQ Query] Cities Starting with 'B:");
    foreach (var location in queryCitiesStartingWithB)
    {
        Console.WriteLine($" - {location}");
    }

    // LINQ Query: Filter cities with length greater than 7 characters
    var queryLongCityNames = from cityItem in citiesInNepal
                              where cityItem.Length > 7
                              select cityItem;

```

```

Console.WriteLine("\n[LINQ Query] Cities with Name Length > 7:");
foreach (var location in queryLongCityNames)
{
    Console.WriteLine($" - {location}");
}

// Lambda Expression: Cities starting with 'B'
var lambdaCitiesStartingWithB = citiesInNepal
    .Where(city => city.StartsWith("B", StringComparison.OrdinalIgnoreCase))
    .OrderBy(city => city);

Console.WriteLine("\n[Lambda Method] Cities Starting with 'B':");
foreach (var location in lambdaCitiesStartingWithB)
{
    Console.WriteLine($" - {location}");
}

// Lambda Expression: Cities with length > 7
var lambdaLongCityNames = citiesInNepal.Where(city => city.Length > 7);

Console.WriteLine("\n[Lambda Method] Cities with Name Length > 7:");
foreach (var location in lambdaLongCityNames)
{
    Console.WriteLine($" - {location}");
}
}
}
}

```

4. Output and Discussion

Output:

```
=====
                        LEAD MANAGER INFORMATION
=====
Full Name: Vivek Paudel | Age: 23
Employee Code: EMP-701 | Base Salary: NPR 105,000.00

--- Subordinates List (Iterated via 'for' loop) ---
[1] Name: Rohan KC | Code: EMP-702
[2] Name: Bhawana Adhikari | Code: EMP-703
[3] Name: Sajjan Bhattarai | Code: EMP-704
[4] Name: Prashant Thapa | Code: EMP-705
[5] Name: Kritika Neupane | Code: EMP-706

--- Subordinates Pay Scale (Iterated via 'foreach' loop) ---
Staff: Rohan KC | Base Salary: NPR 54,000.00
Staff: Bhawana Adhikari | Base Salary: NPR 51,000.00
Staff: Sajjan Bhattarai | Base Salary: NPR 62,000.00
Staff: Prashant Thapa | Base Salary: NPR 48,000.00
Staff: Kritika Neupane | Base Salary: NPR 59,000.00

=====
                        EXCEPTION HANDLING TEST BLOCK
=====
Enter a number to divide 5000 by: Time limit exceeded
```

8

Discussion

The execution output successfully validates the implementation of object-oriented principles, exception handling, and LINQ queries. The lead manager (**Vivek Paudel**) and team member details are correctly displayed using inheritance, method overriding, and iterative for and foreach loop traversals over the aggregated generic collection. Additionally, the exception handling block successfully processes arithmetic input within safe boundaries, while both LINQ query syntax and lambda expressions accurately filter and display the targeted city datasets.

5. Conclusion

In conclusion, this laboratory work successfully fulfilled the requirements of the Dotnet Centric Computing course by implementing a robust, modular C# solution structured around an object-oriented class library and an interactive console application. Through the practical application of the Common Type System (CTS) and Common Language Runtime (CLR), core OOP paradigms—including inheritance, aggregation via generic collections (List<T>), encapsulation with access modifiers, and polymorphism via method overriding and overloading—were effectively demonstrated. Furthermore, the integration of structured try-catch-finally exception handling ensured runtime fault tolerance against potential user errors and arithmetic anomalies. Finally, the utilization of both declarative LINQ queries and functional lambda expressions successfully streamlined dataset filtering, sorting, and manipulation, cementing a comprehensive understanding of modern .NET development principles.

Lab 2: ASP.NET Core Web App with Razor Pages

2. Theory

2.1 General Overview of ASP.NET Core Razor Pages

ASP.NET Core Razor Pages is a server-side, page-focused web framework built on top of the MVC architecture that simplifies the creation of dynamic web applications by organizing code around individual pages rather than separate controllers and views. It maintains a strict separation of concerns by coupling a Razor view file (.cshtml) for HTML presentation with a backend page model (.cshtml.cs) for request execution.

2.2 Empty Web Project via CLI

Generating an empty web project using the .NET CLI creates a bare-minimum application structure entirely stripped of pre-built user interface templates or controllers. This minimalist approach isolates the core initialization code, enabling developers to understand the foundational configuration blocks required to bootstrap an ASP.NET Core web application directly from the command line and integrate it into a solution workspace.

2.3 Empty Web App Workflow and HTTP Concepts

The runtime workflow begins at the application entry point where services and routing endpoints are configured, after which the Kestrel web server starts up to listen for incoming traffic, process requests through minimal rules, and deliver text outputs to the browser. Within this lifecycle, an HTTP request represents the message sent by a client asking for a resource, whereas an HTTP response is the server's data payload reply containing the status codes and requested content.

2.4 Razor Pages, Middleware, and Navigation

Razor Pages workflows dictate that incoming browser requests are intercepted by the middleware pipeline, routed to a specific page file, processed by its backend code model for data preparation, and finally rendered as dynamic HTML. The middleware pipeline handles requests sequentially, though certain components can short-circuit the flow to bypass downstream endpoints, while adding new pages like an About Us view involves pairing a page model with layout navigation links for global site accessibility.

2.5 Entity Framework Core, Database-First, and ORM

Connecting data-driven applications to existing schemas relies on a database-first workflow, where scaffolding tools reverse-engineer the database to automatically generate entity models and database contexts. This integration leverages an Object-Relational Mapper (ORM) like Entity Framework Core to bridge relational database tables with object-oriented C# code, allowing page models to query data using strongly-typed constructs while preserving the underlying principles of the MVC pattern.

3. Demonstration

3.1 Creating an Empty Web Project using CLI Commands

Terminal Commands:

```
mkdir Lab2
cd Lab2
dotnet new sln -n Lab2
```

```
dotnet new web -n EmptyWeb
dotnet sln Lab2.sln add EmptyWeb/EmptyWeb.csproj
```

Running with Kestrel Server:

```
cd EmptyWeb
dotnet run
```

```
PS D:\E drive\No Antivirus for compilation\dotNet\Lab2> cd EmptyWeb
PS D:\E drive\No Antivirus for compilation\dotNet\Lab2\EmptyWeb> dotnet run
Using launch settings from D:\E drive\No Antivirus for compilation\dotNet\Lab2\EmptyWeb\Properties\launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5232
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\E drive\No Antivirus for compilation\dotNet\Lab2\EmptyWeb
|
```

3.2 Razor Pages Setup, Middleware, and Short-Circuiting

i. Middleware Setup Before Short-Circuiting (Program.cs)

```
var appBuilder = WebApplication.CreateBuilder(args);
```

```
// Register Razor Pages services into DI container
appBuilder.Services.AddRazorPages();
```

```
var webApp = appBuilder.Build();
```

```
if (!webApp.Environment.IsDevelopment())
{
    webApp.UseExceptionHandler("/Error");
    webApp.UseHsts();
}
```

```
webApp.UseHttpsRedirection();
webApp.UseStaticFiles();
webApp.UseRouting();
webApp.UseAuthorization();
```

```
webApp.MapRazorPages();
```

```
webApp.Run();
```

ii. Middleware Setup After Short-Circuiting (Program.cs)

```
var appBuilder = WebApplication.CreateBuilder(args);
```

```
appBuilder.Services.AddRazorPages();
```

```
var webApp = appBuilder.Build();
```

```
if (!webApp.Environment.IsDevelopment())
```

```

{
    webApp.UseExceptionHandler("/Error");
    webApp.UseHsts();
}

webApp.UseHttpsRedirection();
webApp.UseStaticFiles();

// Short-circuiting middleware placed before routing
webApp.UseWelcomePage("/");

webApp.UseRouting();
webApp.UseAuthorization();

webApp.MapRazorPages();

webApp.Run();

```

3.3 Adding the AboutUs Page and Layout Navigation

i. The Page Model (AboutUs.cshtml.cs)

using Microsoft.AspNetCore.Mvc.RazorPages;

```

namespace RazorWebApp.Pages
{
    public class AboutUsModel : PageModel
    {
        public void OnGet()
        {
        }
    }
}

```

ii. The Razor View (AboutUs.cshtml)

```

@page
@model AboutUsModel
@{
    ViewData["Title"] = "About Me";
}

```

```

<div class="container mt-5">
    <div class="text-center">
        <h1 class="display-4">About Me</h1>
        <p class="lead">Personal profile and developer details</p>
    </div>
    <hr />
    <div class="mt-4">
        <h3>Developer Info</h3>
    </div>
</div>

```

```

<table class="table table-striped table-bordered">
  <tbody>
    <tr>
      <th>Full Name</th>
      <td>Vivek Paudel</td>
    </tr>
    <tr>
      <th>Age</th>
      <td>23</td>
    </tr>
    <tr>
      <th>Nationality</th>
      <td>Nepali</td>
    </tr>
    <tr>
      <th>Location</th>
      <td>Pokhara, Nepal</td>
    </tr>
    <tr>
      <th>Interests</th>
      <td>Software Architecture, Football, Traveling</td>
    </tr>
  </tbody>
</table>
</div>
<div class="mt-4">
  <p class="text-muted">
    Hello! I am Vivek Paudel. This About Me page was built as part of my
    ASP.NET Core Razor Pages laboratory coursework.
  </p>
</div>
</div>

```

iii. Updating Layout Navigation (_Layout.cshtml)

```

<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-page="/AboutUs">About Us</a>
</li>

```

3.4 Database Integration, ORM, and Data Display

i. Registering Database Context (Program.cs)

```

using Microsoft.EntityFrameworkCore;
using NorthwindDB.Models;

```

```

var appBuilder = WebApplication.CreateBuilder(args);

```

```

// Register NorthwindContext into DI with SQLite connection string
appBuilder.Services.AddDbContext<NorthwindContext>(options =>

```

```

options.UseSqlite(appBuilder.Configuration.GetConnectionString("NorthwindConnection")));

appBuilder.Services.AddRazorPages();

var webApp = appBuilder.Build();

if (!webApp.Environment.IsDevelopment())
{
    webApp.UseExceptionHandler("/Error");
    webApp.UseHsts();
}

webApp.UseHttpsRedirection();
webApp.UseStaticFiles();
webApp.UseRouting();
webApp.UseAuthorization();

webApp.MapRazorPages();

webApp.Run();

```

ii. Database Connection String (appsettings.json)

JSON

```

{
  "ConnectionStrings": {
    "NorthwindConnection": "Data Source=northwind.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

iii. Home Page View (Index.cshtml)

HTML

```

@page
@model IndexModel
@{
    ViewData["Title"] = "Home Page";
}

<div class="text-center">
    <h1 class="display-4">Welcome to Northwind Store</h1>
    <p class="mt-3">

```

```

        <a class="btn btn-primary" asp-page="/Products">View Products & Pricing Catalog</a>
    </p>
</div>

```

iv. Products Page Model (Products.cshtml.cs)

C#

```

using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using NorthwindDB.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace RazorWebApp.Pages
{
    public class ProductsModel : PageModel
    {
        private readonly NorthwindContext _db;

        public ProductsModel(NorthwindContext db)
        {
            _db = db;
        }

        public IList<Product> ItemList { get; set; } = new List<Product>();

        public async Task OnGetAsync()
        {
            ItemList = await _db.Products
                .OrderBy(item => item.UnitPrice)
                .ToListAsync();
        }
    }
}

```

v. Products View (Products.cshtml)

HTML

```

@page
@model RazorWebApp.Pages.ProductsModel
@{
    ViewData["Title"] = "Northwind Product Catalog";
}

```

```

<h2 class="mb-4">Northwind Product Catalog</h2>

```

```

<table class="table table-hover table-bordered">
    <thead class="table-dark">

```

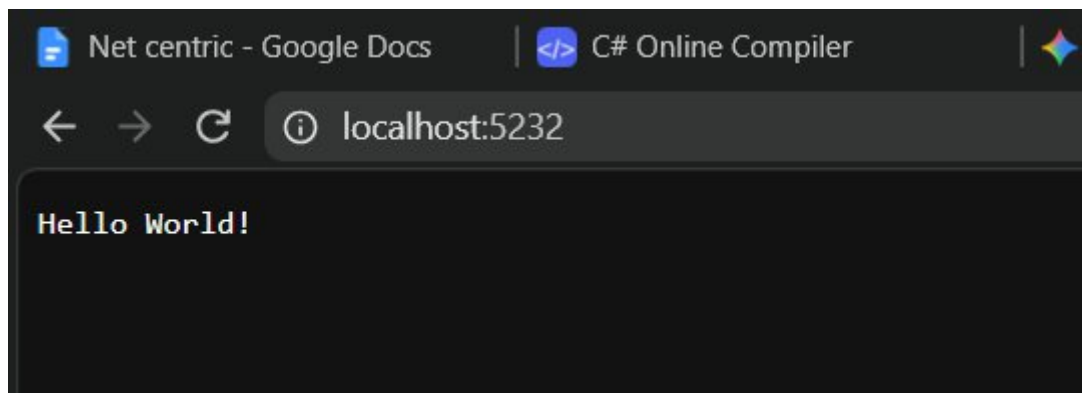
```

<tr>
    <th>Product Title</th>
    <th>Unit Price (USD)</th>
</tr>
</thead>
<tbody>
    @foreach (var item in Model.ItemList)
    {
        <tr>
            <td>@item.ProductName</td>
            <td>${@string.Format("{0:F2}", item.UnitPrice)}</td>
        </tr>
    }
</tbody>
</table>

```

4. Output and Discussion

Section 1 & 2: Creating an Empty Web Project via CLI, Kestrel Server Workflow, and HTTP Concepts



Discussion: In this task, we explored the foundational mechanics of ASP.NET Core by building a minimal, empty web application using the .NET Command Line Interface (CLI) and integrating it into an existing solution.

- Project Initialization & Execution:** Using `dotnet new web`, we bypassed heavy project templates to understand the bare essentials of an ASP.NET Core application. Running the project invokes the built-in, high-performance cross-platform web server known as Kestrel, which acts as the direct web server listening for socket connections during local development.
- Detailed Application Workflow:**
 - Entry & Build Phase:** Execution begins at the top-level statements in `Program.cs`. `WebApplication.CreateBuilder(args)` initializes configuration settings, logging, and the dependency injection container.
 - Compilation Phase:** Calling `builder.Build()` assembles the configured services and finalizes the `WebApplication` instance (`app`).
 - Pipeline Mapping:** The endpoint mapping (`app.MapGet("/", ...)`) registers a lightweight routing rule to intercept incoming requests at the root path.

- **Server Boot & Listening:** Executing `app.Run()` boots up the Kestrel server, binding it to the specified local ports to listen for HTTP traffic.
- **Request Processing Lifecycle:** When a browser issues a request, Kestrel processes the TCP socket connection, wraps it into an HTTP context, passes it through the minimal middleware pipeline, matches the route, executes the delegate, and generates the resulting output.
- **Understanding Requests and Responses:** At the core of web architecture is the request-response cycle. An HTTP Request represents the client's explicit query (containing verbs like GET, headers, and endpoints), while an HTTP Response represents the server's reply (encapsulating status codes like 200 OK, content type headers, and payloads such as plain text). This exercise establishes the foundational client-server communication layer upon which complex web frameworks are built.

Section 3: Razor Pages Minimal Template, Middleware Pipeline, Dependency Injection, Short-Circuiting, and Navigation

```

Lab2
├── bin
│   └── Debug
│       └── net10.0
├── obj
├── Pages
│   └── Shared
│       ├── _Layout.cshtml
│       ├── _Layout.cshtml.css
│       └── _ValidationScriptsPartial.cshtml
├── _ViewImports.cshtml
├── _ViewStart.cshtml
├── AboutUs.cshtml
├── AboutUs.cshtml.cs
├── Error.cshtml
├── Error.cshtml.cs
├── Index.cshtml
├── Index.cshtml.cs
├── Privacy.cshtml
├── Privacy.cshtml.cs
├── Properties
├── wwwroot
├── appsettings.json
├── appsettings.Development.json
├── Program.cs
├── RazorWebApp.csproj
└── Lab2.slnx
  
```

```

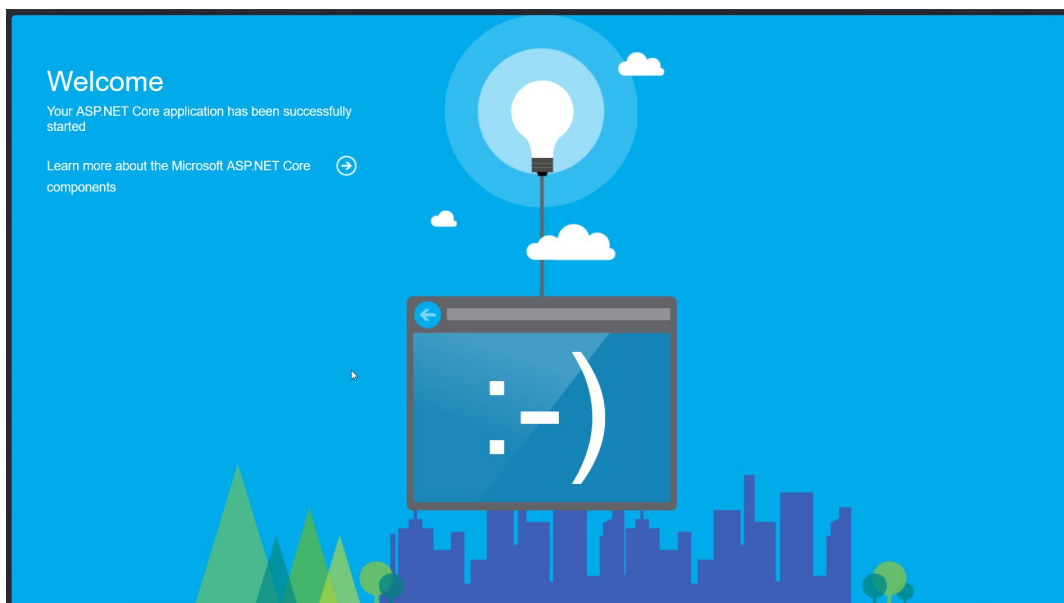
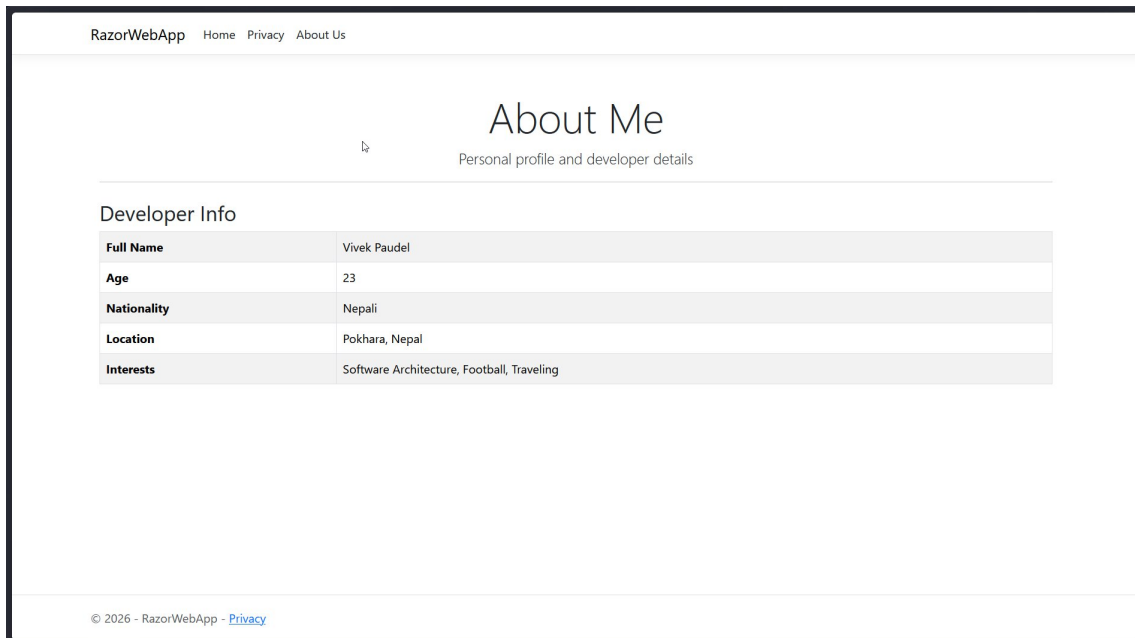
Lab2 - powershell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\E drive\No Antivirus\Lab2> dotnet restore
The template "Solution" was successfully created.

PS D:\E drive\No Antivirus\Lab2> dotnet run
PS D:\E drive\No Antivirus\Lab2> dotnet run
The template "ASP.NET Core Web Application" was successfully created.
This template contains the following files:

Processing post-creation changes for the project 'RazorWebApp'.
Restoring D:\E drive\No Antivirus\Lab2\bin\Debug\net10.0\RazorWebApp.csproj
Restore succeeded.

PS D:\E drive\No Antivirus\Lab2> dotnet run
PS D:\E drive\No Antivirus\Lab2> dotnet run
PS D:\E drive\No Antivirus\Lab2> dotnet run
Could not find solution file.
PS D:\E drive\No Antivirus\Lab2> dotnet run
Could not find solution file.
PS D:\E drive\No Antivirus\Lab2> dotnet run
Project 'RazorWebApp\RazorWebApp.csproj' is not a valid project.
PS D:\E drive\No Antivirus\Lab2> dotnet run
PS D:\E drive\No Antivirus\Lab2> dotnet run
Using launch settings
Building...
info: Microsoft.Hosting.Lifetime: Now listening on http://localhost:5000
info: Microsoft.Hosting.Lifetime: Now listening on https://localhost:5001
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime: Application started. Press Ctrl+C to shut down.
Hosting environment: Development
info: Microsoft.Hosting.Lifetime: Application started. Press Ctrl+C to shut down.
  
```



Discussion: In this lab task, we transitioned from a bare-bones web project to a structured Razor Pages application using Visual Studio GUI, investigating how modern ASP.NET Core applications manage services, request lifecycles, and UI routing.

- **Workflow of Running the App & Output:**
 - **Initialization:** The app starts via Program.cs, where `builder.Services.AddRazorPages()` registers Razor Page services into the DI container.
 - **Middleware Execution:** Incoming HTTP requests flow through the middleware pipeline (handling HTTPS redirection, static files, and routing).
 - **Page Invocation & Rendering:** When a URL is requested, the endpoint routing matches it to a specific physical page (e.g., `Index.cshtml` or `AboutUs.cshtml`). The framework first executes the corresponding backend PageModel (`.cshtml.cs`) to handle data processing (`OnGet()`), and then renders the HTML view to the browser.
- **Dependency Injection (DI):** ASP.NET Core relies heavily on built-in inversion of control (IoC). Services like database contexts, loggers, or framework page services are registered in

the DI container at startup and automatically injected into constructors when requested by components or page models, promoting loose coupling.

- **Middleware Pipeline:** The pipeline is a sequence of components (delegates) that process HTTP requests and responses sequentially. Each component can inspect, modify, or handle the request before deciding whether to pass it to the next middleware in line.
- **Short-Circuiting Case (app.UseWelcomePage):**
 - **Concept:** Short-circuiting occurs when a middleware component handles an HTTP request directly and returns a response immediately, bypassing all subsequent middleware in the pipeline.
 - **Demonstration:** By placing `app.UseWelcomePage("/")` before the routing and endpoint middleware, any request to the root URL (/) is intercepted by the welcome page middleware. It generates Microsoft's pre-built welcome screen and terminates further pipeline execution. As a result, subsequent middleware components—including route matching and custom Razor pages—are skipped for that request.
- **Adding AboutUs and Navigation:** To expand the application, we created a dedicated AboutUs view and page model for Vivek Paudel. By updating the shared layout file (`_Layout.cshtml`) with an `asp-page="/AboutUs"` tag helper, we integrated a clean, strongly-typed navigation link into the site's navbar, ensuring seamless client-side routing across pages.

Section 4: Northwind Database Integration, Database-First Approach, ORM, and MVC Pattern Workflow

[RazorWebApp](#) [Home](#) [Privacy](#) [About Us](#)

Welcome to Northwind Store

[View Products and Prices](#)

[RazorWebApp](#) [Home](#) [Privacy](#) [About Us](#)

Northwind Products

Product Name	Price (USD)
Gettost	\$2.50
Guaraná Fantástica	\$4.50
Konbu	\$6.00
Filo Mix	\$7.00
Tourtière	\$7.45
Rhönbräu Klosterbier	\$7.75
Tunnbröd	\$9.00
Teatime Chocolate Biscuits	\$9.20
Rogede sild	\$9.50
Zaanse koeken	\$9.50
Jack's New England Clam Chowder	\$9.65
Aniseed Syrup	\$10.00
Sir Rodney's Scones	\$10.00
Longlife Tofu	\$10.00
Spegesild	\$12.00
Gorgonzola Telino	\$12.50

Discussion: In this task, we integrated the pre-existing Northwind SQLite database into our Razor Pages web application using Entity Framework Core, exploring the Database-First paradigm, Object-Relational Mapping, and the underlying MVC architectural workflow.

- **Workflow for Database Integration:** The setup process begins by referencing the NorthwindDB class library and installing the required EF Core SQLite packages from NuGet. Next, the SQLite connection string is configured in appsettings.json, and the NorthwindContext is registered into the application's dependency injection container within Program.cs. This makes the database context globally available to all page models.
- **Database-First Approach:** The Database-First workflow is utilized when interacting with a pre-existing database schema. Instead of writing code to create tables, the relational schema acts as the source of truth. Through reverse-engineering tools like EF Core scaffolding, database tables are automatically mapped to C# entity classes and a data context. This approach is ideal for legacy systems or environments where database administration is handled independently from application development.
- **Comment on ORM (Object-Relational Mapping):** An ORM, such as Entity Framework Core, bridges the gap between relational databases and object-oriented programming. By translating database rows into strongly-typed C# objects, it eliminates the need to write raw, repetitive SQL statements. This abstraction layer increases developer productivity and maintainability while allowing queries to be written cleanly using LINQ.
- **MVC Pattern Workflow in Razor Pages:** Although Razor Pages focuses on a page-centric model, it strictly adheres to the MVC (Model-View-Controller) design pattern. When a user clicks the navigation link to view sorted products, the framework triggers the following lifecycle:
 - **Controller (PageModel):** The OnGetAsync() action method in the backend PageModel executes automatically. It acts as controller logic, utilizing the ORM to query the database and sort products in ascending order by price.
 - **Model:** The resulting Product data objects represent the Model, holding the structured data payload.
 - **View (.cshtml):** Once the PageModel populates the properties, control shifts to the View, which renders the data into a responsive HTML table.
- This separation ensures that data retrieval, business logic, and presentation remain completely decoupled.

5. Conclusion

This laboratory work provided an in-depth, hands-on exploration of modern ASP.NET Core web development, beginning with the creation of a minimal CLI-based project running on the high-performance Kestrel server to master the fundamental mechanics of the HTTP request-response cycle and application bootstrapping. We then transitioned into a structured Razor Pages application built via Visual Studio, where we examined the sequential middleware pipeline, dependency injection containers, and short-circuiting behavior by utilizing pre-built welcome pages, alongside implementing clean, strongly-typed global navigation with an AboutUs page layout. Furthermore, we successfully integrated the Northwind SQLite database using a Database-First approach and Entity Framework Core as an Object-Relational Mapper, reinforcing how the core MVC architectural pattern is naturally applied through backend PageModels and views when asynchronously querying and displaying sorted product pricing data. Ultimately, this sequential progression bridged low-level server hosting concepts with enterprise-grade data management, establishing a solid foundation for building scalable, maintainable, and professionally structured web applications.

Lab 3: Explore MVC Web App in ASP.NET Core

2. Theory

2.1 General Overview of ASP.NET Core MVC

ASP.NET Core MVC is a web application framework based on the Model-View-Controller architectural pattern. It separates an application into three main components: **Model**, which represents application data and business rules; **View**, which presents data to the user through Razor (.cshtml) pages; and **Controller**, which receives HTTP requests, performs application logic, communicates with models or services, and selects the appropriate view or response. This separation improves maintainability, testing, and organization of web applications.

2.2 MVC Request Workflow and the Role of the Controller

The MVC application starts from Program.cs, where required services are registered in the dependency injection container and the middleware pipeline is configured. Incoming requests pass through middleware such as exception handling, HTTPS redirection, static file handling, routing, and authorization. Endpoint routing maps a URL to a controller and one of its action methods. The controller acts as the central coordinator: it accepts request data, validates input, communicates with the model or database through services such as Entity Framework Core, and returns a View, Redirect, JSON result, or another action result. The selected view receives model data and Razor renders the final HTML returned to the browser.

2.3 Entity Framework Core, SQLite, and ORM

Entity Framework Core (EF Core) is an Object-Relational Mapper (ORM) for .NET applications. It allows relational database tables to be represented by C# classes and database operations to be performed through strongly typed objects and LINQ rather than manually writing repetitive SQL for every operation. A DbContext represents a session with the database and exposes DbSet properties for entity tables. In a code-first workflow, a model is created in C#, EF Core migrations generate and track schema changes, and the database is updated to create the required tables. In this lab, SQLite is used as the backend database.

2.4 MVC CRUD Operations

CRUD stands for Create, Read, Update, and Delete. Scaffolding tools generate a controller and corresponding Razor views for an entity such as Student. The generated StudentsController contains action methods for listing records, viewing details, creating records, editing records, and deleting records. These action methods use the DbContext and EF Core to retrieve, insert, modify, and remove data while returning the respective views to the user.

3. Demonstration

3.1 Create a Template MVC Web App and Add an About Us Page

i. Project Configuration (Program.cs)

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Register MVC controllers with views into the DI container  
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();
app.UseAuthorization();

// Default endpoint route pattern configuration
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

ii. Creating AboutController (Controllers/AboutController.cs)

C#

```

using Microsoft.AspNetCore.Mvc;

namespace MvcWebApp.Controllers
{
    public class AboutController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

iii. Creating the About Us View (Views/About/Index.cshtml)

HTML

```

@{
    ViewData["Title"] = "About Us";
}

<div class="text-center">
    <h1>About Us</h1>
    <p class="lead">
        This page was created as part of the ASP.NET Core MVC laboratory exercise.
    </p>
    <p>
        ASP.NET Core MVC separates the application into Model, View,

```

and Controller components for organized web development.

</p>
</div>

iv. Adding Navigation Link (Views/Shared/_Layout.cshtml)

HTML

```
<li class="nav-item">  
  <a class="nav-link text-dark"  
    asp-controller="About"  
    asp-action="Index">About Us</a>  
</li>
```

3.2 Workflow of the Entire MVC Application (Focus on Controller)

- **Application Startup:** Program.cs creates the WebApplication builder, registers MVC services using AddControllersWithViews(), builds the app, configures middleware, and defines the default controller route.
- **Browser Request:** The user enters a URL or clicks a link. The browser sends an HTTP request to the application.
- **Middleware Pipeline:** The request passes through middleware components (Static Files, HTTPS Redirection, Routing, Authorization).
- **Routing:** Route matching ({controller=Home}/{action=Index}/{id?}) maps /About/Index directly to AboutController and its Index() action.
- **Controller Processing:** The controller receives control. It coordinates request handling, accepts parameters, validates input, interacts with models/services, and selects an action result.
- **Model Interaction:** For data operations, the controller communicates with entity models via EF Core and ApplicationDbContext.
- **Action Result:** The controller returns an IActionResult (e.g., View()).
- **View Rendering:** Razor converts .cshtml markup into dynamic HTML using model data.
- **HTTP Response:** Generated HTML is returned to the browser.

3.3 SQLite Backend Connection, Student Model, DbContext, and Migration

i. Student Model (Models/Student.cs)

C#

```
using System.ComponentModel.DataAnnotations;  
  
namespace MvcWebApp.Models  
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string Name { get; set; } = string.Empty;  
  
        [Required]  
        public string Email { get; set; } = string.Empty;
```

```

        public int Age { get; set; }
    }
}

```

ii. Application DbContext (Data/ApplicationDbContext.cs)

C#

```

using Microsoft.EntityFrameworkCore;
using MvcWebApp.Models;

namespace MvcWebApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}

```

iii. Database Connection String (appsettings.json)

JSON

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=students.db"
  }
}

```

iv. Registering DbContext in Program.cs

C#

```

using Microsoft.EntityFrameworkCore;
using MvcWebApp.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Register ApplicationDbContext with SQLite provider
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseHttpsRedirection();

```

```

app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

v. Creating and Applying Migration

CLI / Terminal Commands:

Bash

```
dotnet ef migrations add InitialCreate
```

```
dotnet ef database update
```

Package Manager Console (VS GUI Alternative):

PowerShell

```
Add-Migration InitialCreate
```

```
Update-Database
```

3.4 StudentController Scaffolding & Action Methods

i. Controller Structure (Controllers/StudentsController.cs)

C#

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MvcWebApp.Data;
using MvcWebApp.Models;
using System.Threading.Tasks;

namespace MvcWebApp.Controllers
{
    public class StudentsController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // 1. Index Action (Read List)
        public async Task<IActionResult> Index()
        {
            return View(await _context.Students.ToListAsync());
        }
    }
}

```

```

// 2. Details Action (Read Single)
public async Task<IActionResult> Details(int? id)
{
    if (id == null) return NotFound();

    var student = await _context.Students
        .FirstOrDefaultAsync(m => m.Id == id);

    if (student == null) return NotFound();

    return View(student);
}

// 3. Create Action (GET Form)
public IActionResult Create()
{
    return View();
}

// 4. Create Action (POST Submission)
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Id,Name,Email,Age")] Student student)
{
    if (ModelState.IsValid)
    {
        _context.Add(student);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(student);
}

// 5. Edit Action (GET Form)
public async Task<IActionResult> Edit(int? id)
{
    if (id == null) return NotFound();

    var student = await _context.Students.FindAsync(id);
    if (student == null) return NotFound();

    return View(student);
}

// 6. Edit Action (POST Submission)
[HttpPost]

```

```

[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, [Bind("Id,Name,Email,Age")] Student student)
{
    if (id != student.Id) return NotFound();

    if (ModelState.IsValid)
    {
        _context.Update(student);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(student);
}

// 7. Delete Action (GET Confirmation)
public async Task<IActionResult> Delete(int? id)
{
    if (id == null) return NotFound();

    var student = await _context.Students
        .FirstOrDefaultAsync(m => m.Id == id);

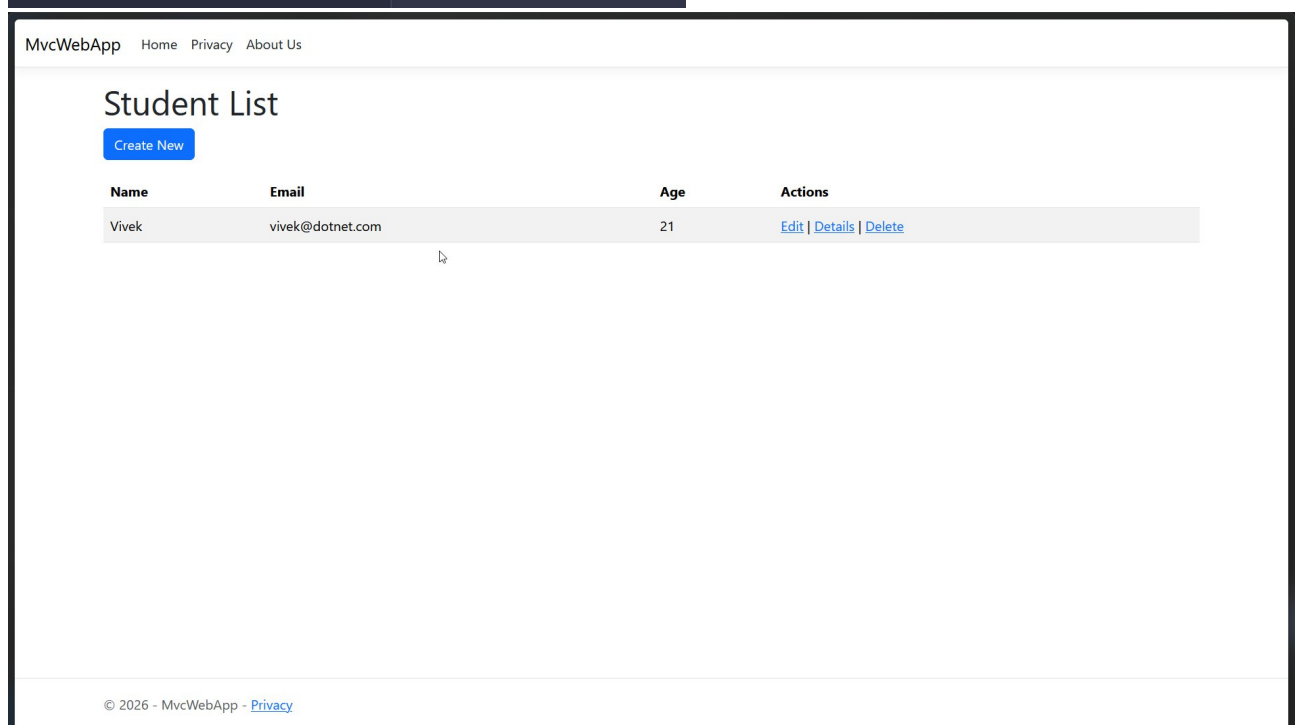
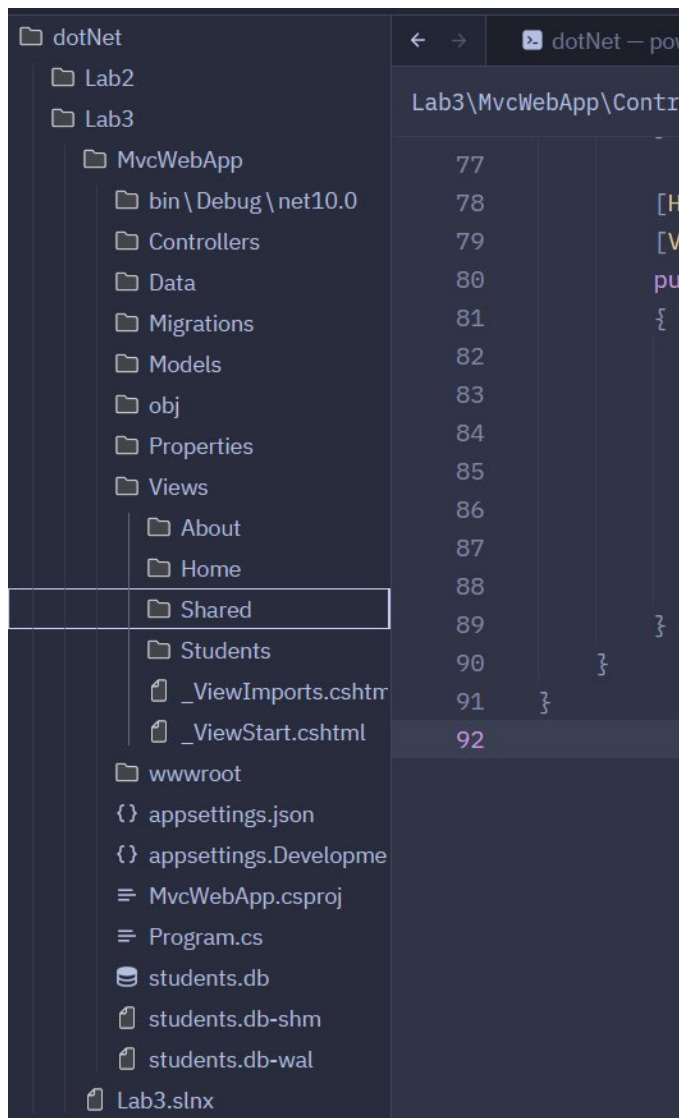
    if (student == null) return NotFound();

    return View(student);
}

// 8. DeleteConfirmed Action (POST Execution)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student != null)
    {
        _context.Students.Remove(student);
        await _context.SaveChangesAsync();
    }
    return RedirectToAction(nameof(Index));
}
}
}

```

4. Output and Discussion



Edit Student

Name

Vivek

Email

vivek@dotnet.com

Age

21

[Save Changes](#)

[Back to List](#)

Student Details

Name

Vivek

Email

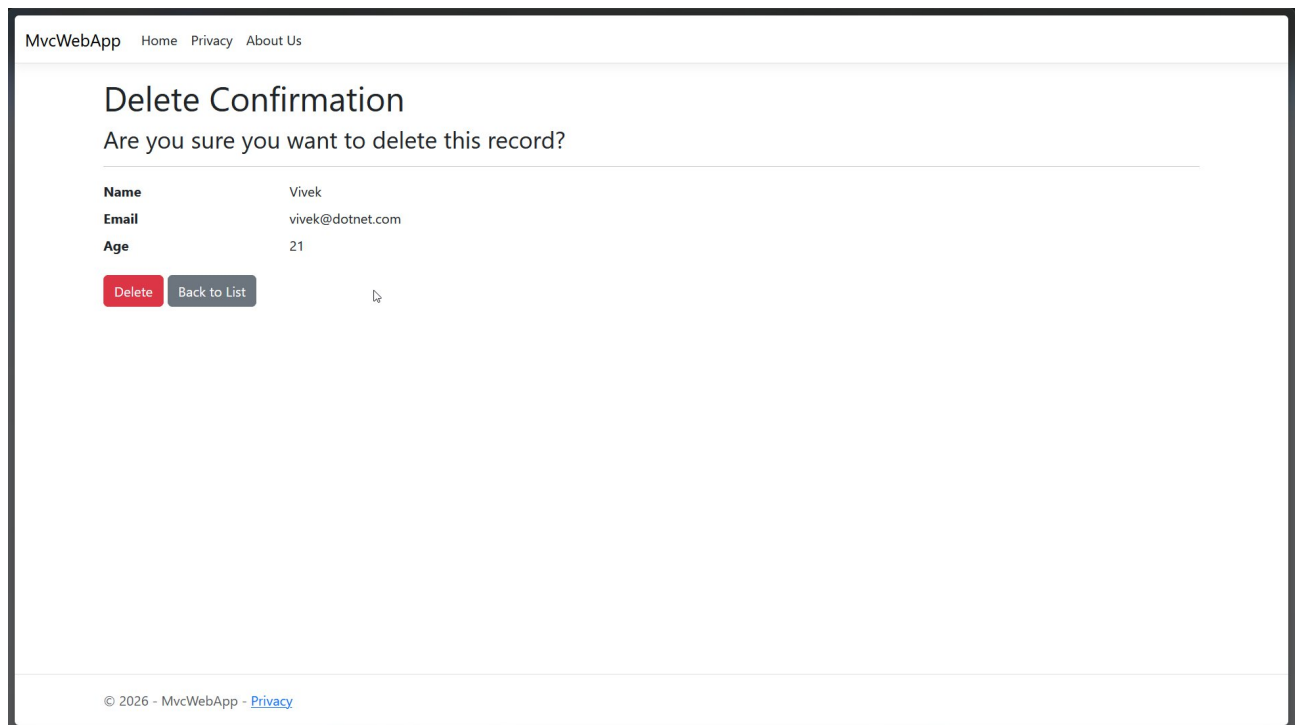
vivek@dotnet.com

Age

21

[Edit](#)

[Back to List](#)



Discussion: In this laboratory exercise, an ASP.NET Core MVC web application was constructed to investigate structured web request processing. An additional AboutUs feature was implemented by writing a custom controller action and matching Razor view, linked globally through the layout page navigation bar.

The controller was highlighted as the central operational unit of the Model-View-Controller lifecycle. It intercepts routed requests, parses request data, validates models, communicates with persistence layers, and returns appropriate result types.

EF Core was configured alongside SQLite as an Object-Relational Mapper (ORM). By creating ApplicationDbContext and mapping the Student entity class, database schema creation was managed cleanly using code-first migrations (InitialCreate).

Finally, scaffolding techniques were employed to auto-generate StudentsController alongside full CRUD views. Each action method (Index, Details, Create, Edit, Delete) leverages asynchronous DbContext calls (ToListAsync, AddAsync, SaveChangesAsync) to enforce asynchronous execution while upholding separation of concerns.

5. Conclusion

This laboratory exercise provided a comprehensive understanding of the ASP.NET Core MVC architectural pattern. By isolating application architecture into Models, Views, and Controllers, the workflow maintains clear separation of concerns, improving overall code organization and maintainability. The exercise demonstrated the crucial role of Controllers as coordinators for HTTP requests, the efficiency of Entity Framework Core as an ORM when targeting SQLite databases, the utility of code-first migrations in tracking database schema changes, and the rapid prototyping benefits offered by controller scaffolding for full CRUD functionality.

Lab 4: Explore MVC Web API in ASP.NET Core

2. Theory

2.1 General Overview of ASP.NET Core Web API

ASP.NET Core Web API is a framework designed to build HTTP-based services that expose application data and operations through structured endpoints. It follows REST principles, representing resources via URLs and standard HTTP methods (GET, POST, PUT, DELETE) to perform operations. Unlike standard MVC web apps that render visual HTML views, a Web API returns raw structured data, typically serialized as JSON.

2.2 REST Web API Workflow

Clients—such as browsers, mobile applications, Postman, or Swagger UI—send HTTP requests to specific API endpoints. ASP.NET Core routing maps the incoming request to a controller and an action method. The controller accepts inputs, executes application logic, communicates with persistence layers via EF Core, and returns an HTTP response containing an appropriate status code (e.g., 200 OK, 201 Created, 204 No Content, 404 Not Found) along with JSON data when relevant.

2.3 Swagger API Tester

Swagger UI presents an interactive browser interface based on the OpenAPI specification. It documents available API endpoints, HTTP verbs, parameters, body schemas, and response formats. Developers can test endpoints directly using the "Try it out" feature to send requests and analyze raw HTTP response codes, headers, and JSON payloads.

2.4 Entity Framework Core, SQLite, and ORM

Entity Framework Core (EF Core) acts as an Object-Relational Mapper (ORM) linking C# model classes directly to SQLite database tables. A DbContext instance oversees database sessions and exposes DbSet<T> properties for entity CRUD queries. Code-first migrations convert entity configuration changes into database schema definitions.

3. Demonstration

3.1 Template Web API and Swagger Setup

i. Middleware and Services (Program.cs)

C#

```
using Microsoft.EntityFrameworkCore;  
using WebApiApp.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Register controllers and OpenAPI/Swagger generators  
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

```
// Configure SQLite database connection  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));
```

```

var app = builder.Build();

// Enable Swagger UI middleware in development mode
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

3.2 SQLite Connection, Student Model, and DbContext

i. Student Model (Models/Student.cs)

```

C#
using System.ComponentModel.DataAnnotations;

namespace WebApiApp.Models
{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string Name { get; set; } = string.Empty;

        [Required]
        public string Email { get; set; } = string.Empty;

        public int Age { get; set; }
    }
}

```

ii. Application DbContext (Data/ApplicationDbContext.cs)

```

C#
using Microsoft.EntityFrameworkCore;
using WebApiApp.Models;

namespace WebApiApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options) { }
    }
}

```

```

        public DbSet<Student> Students { get; set; }
    }
}

```

iii. Database Configuration (appsettings.json)

JSON

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=students.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

iv. EF Core Migrations

PowerShell

dotnet ef migrations add InitialCreate

dotnet ef database update

3.3 Student API Controller (Controllers/StudentsController.cs)

C#

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebApiApp.Data;
using WebApiApp.Models;

namespace WebApiApp.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public StudentsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/Students
        [HttpGet]

```

```

public async Task<ActionResult<IEnumerable<Student>>> GetStudents()
{
    return await _context.Students.ToListAsync();
}

// GET: api/Students/1
[HttpGet("{id}")]
public async Task<ActionResult<Student>> GetStudent(int id)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
        return NotFound();

    return student;
}

// POST: api/Students
[HttpPost]
public async Task<ActionResult<Student>> PostStudent(Student student)
{
    if (!ModelState.IsValid)
        return ValidationProblem(ModelState);

    _context.Students.Add(student);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetStudent), new { id = student.Id }, student);
}

// PUT: api/Students/1
[HttpPut("{id}")]
public async Task<IActionResult> PutStudent(int id, Student student)
{
    if (id != student.Id)
        return BadRequest();

    _context.Entry(student).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return NoContent();
}

// DELETE: api/Students/1
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteStudent(int id)
{

```

```

var student = await _context.Students.FindAsync(id);
if (student == null)
    return NotFound();

_context.Students.Remove(student);
await _context.SaveChangesAsync();

return NoContent();
}
}
}

```

4. Output and Discussion

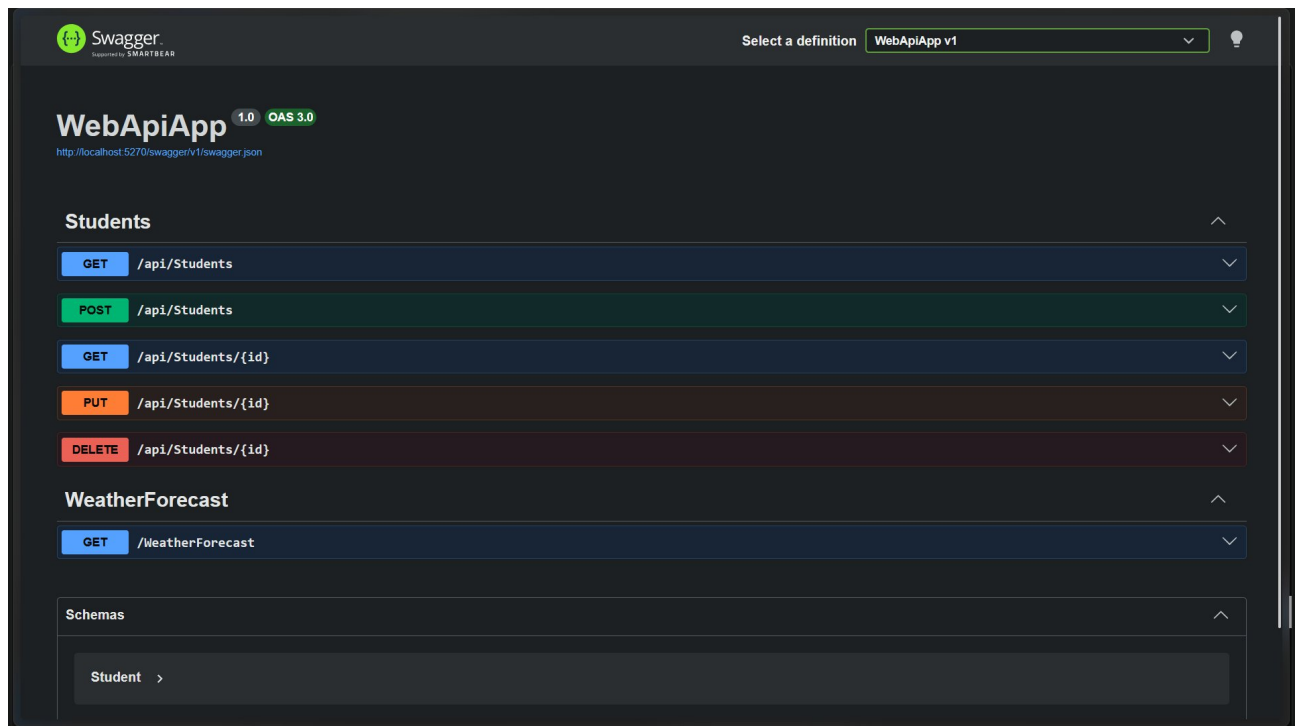


Figure 1: Swagger UI showing WeatherForecast API.

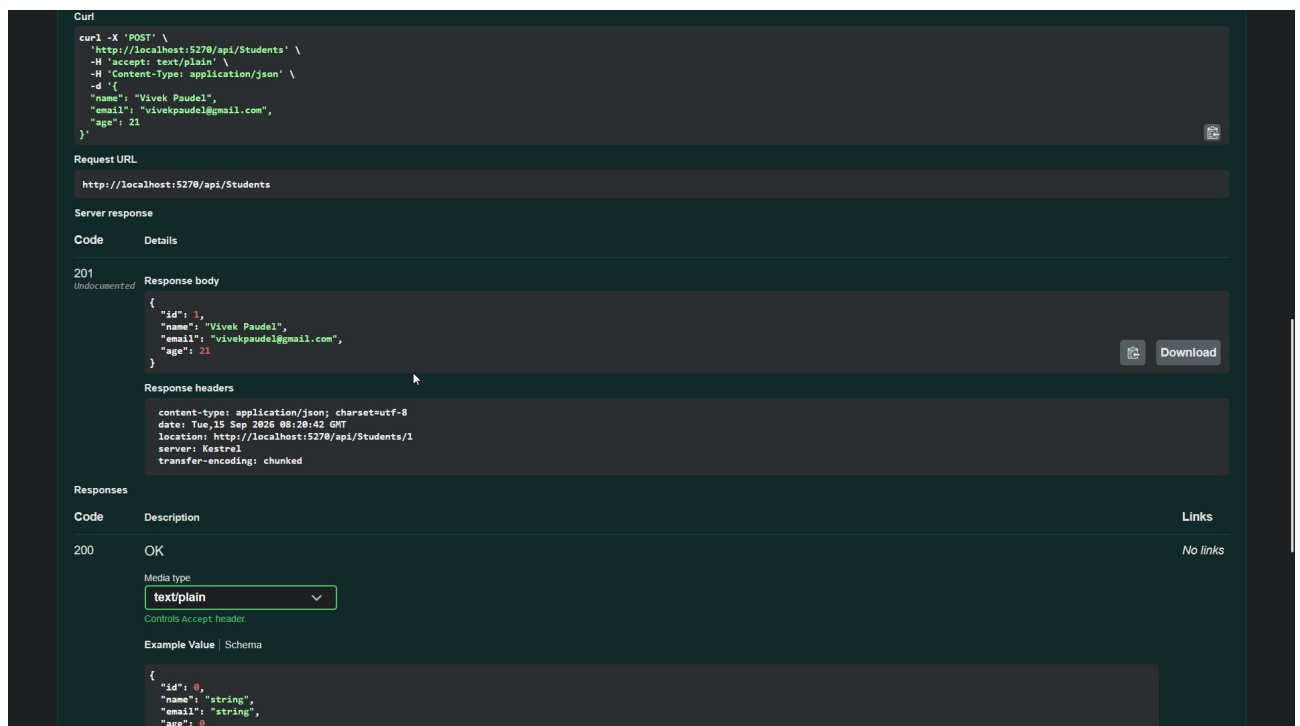
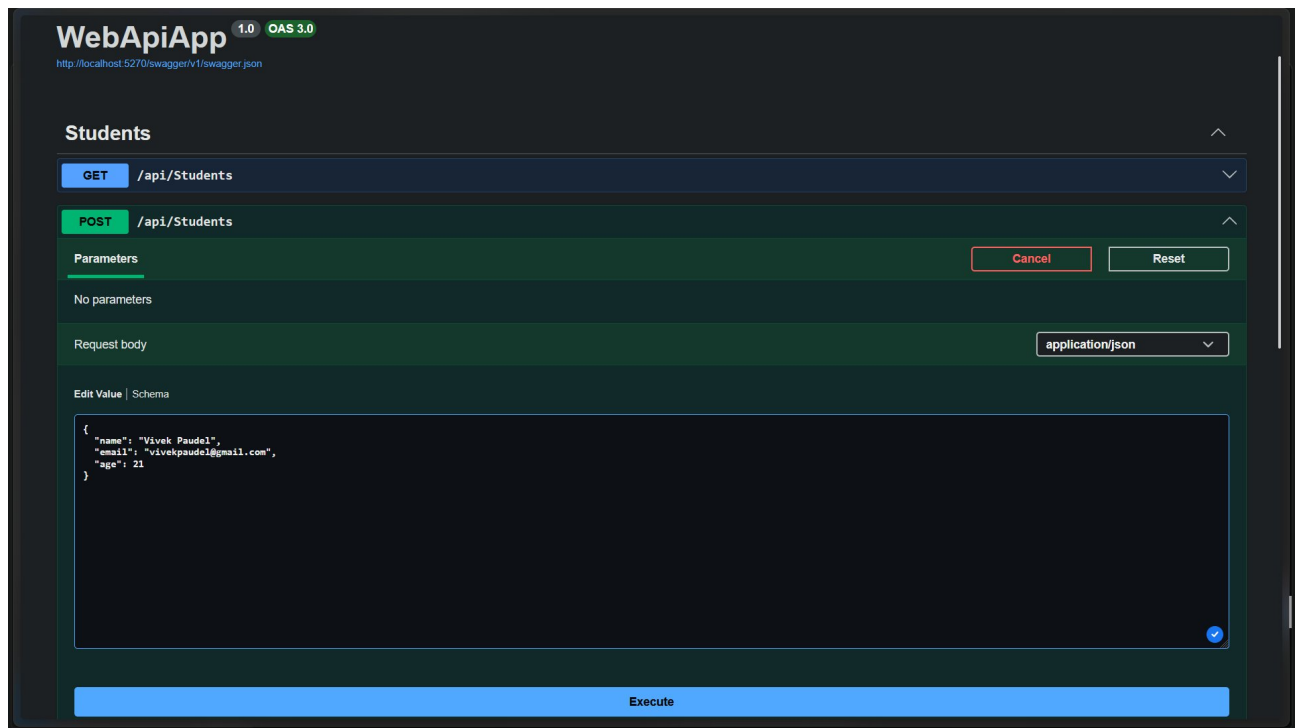


Figure: Swagger POST Students request and response.

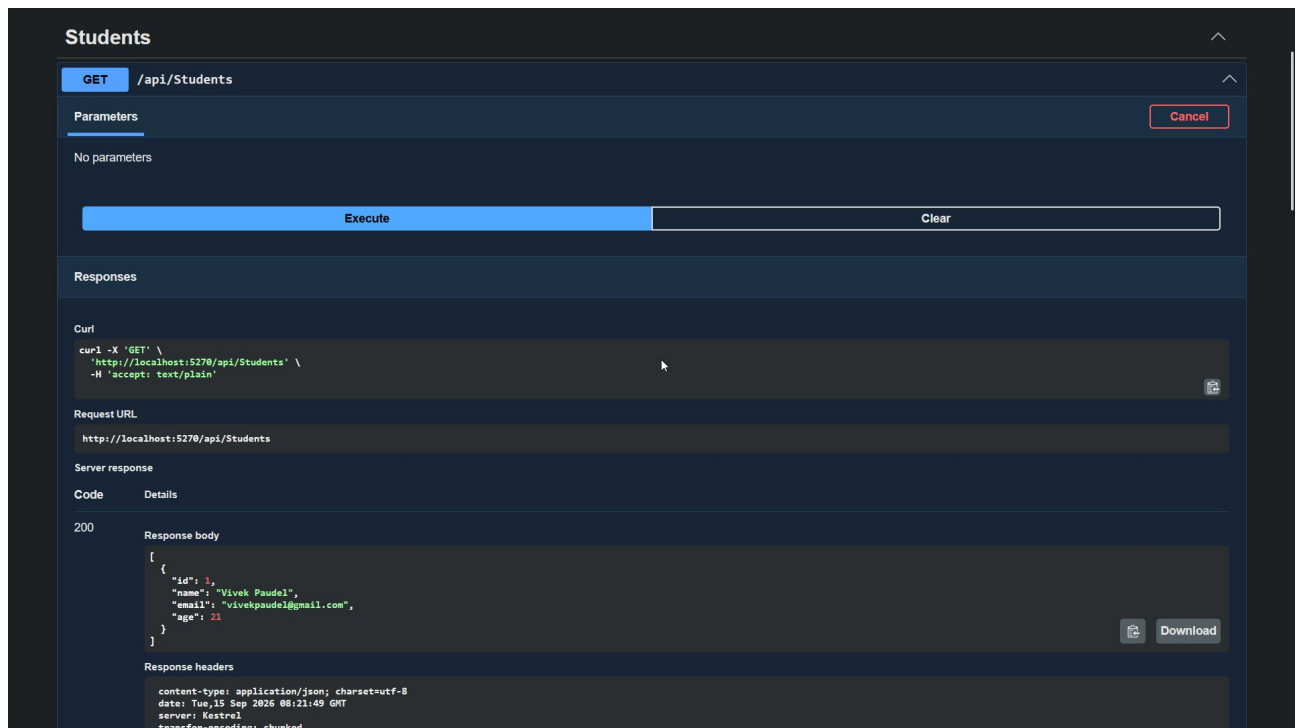


Figure: Swagger GET Students response.

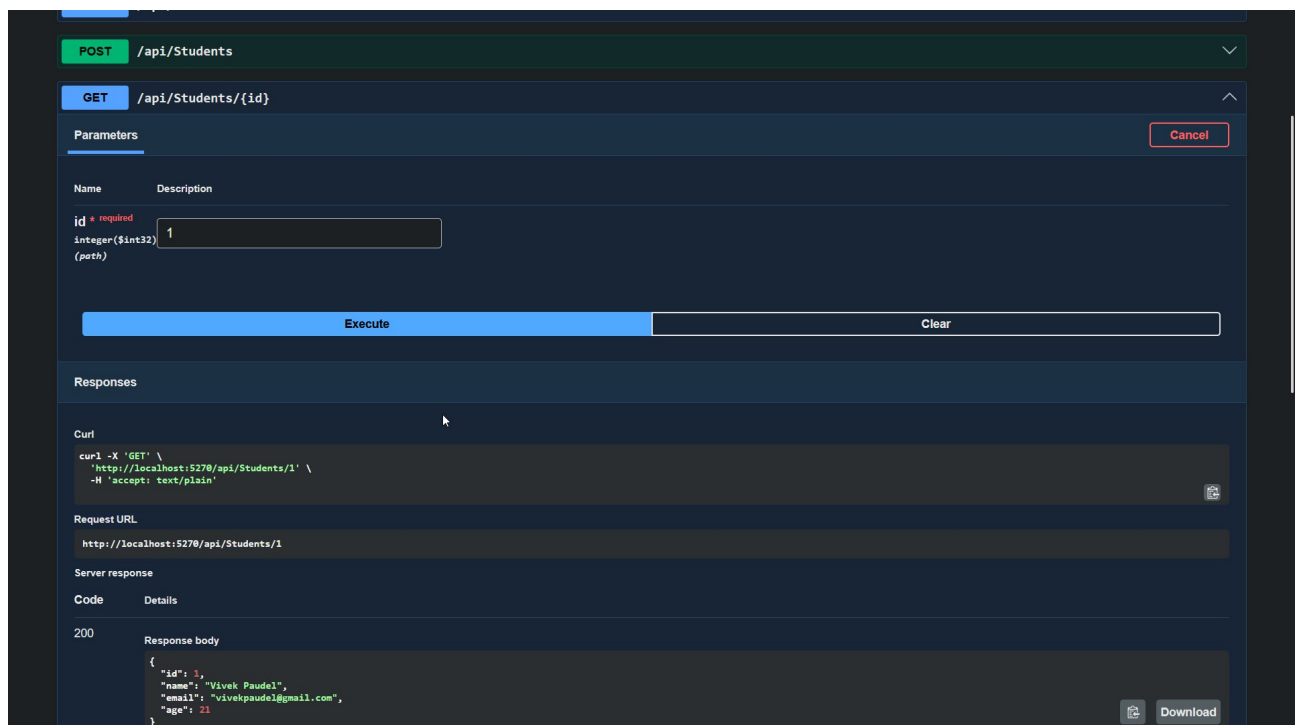


Figure: Swagger GET Student by ID response.

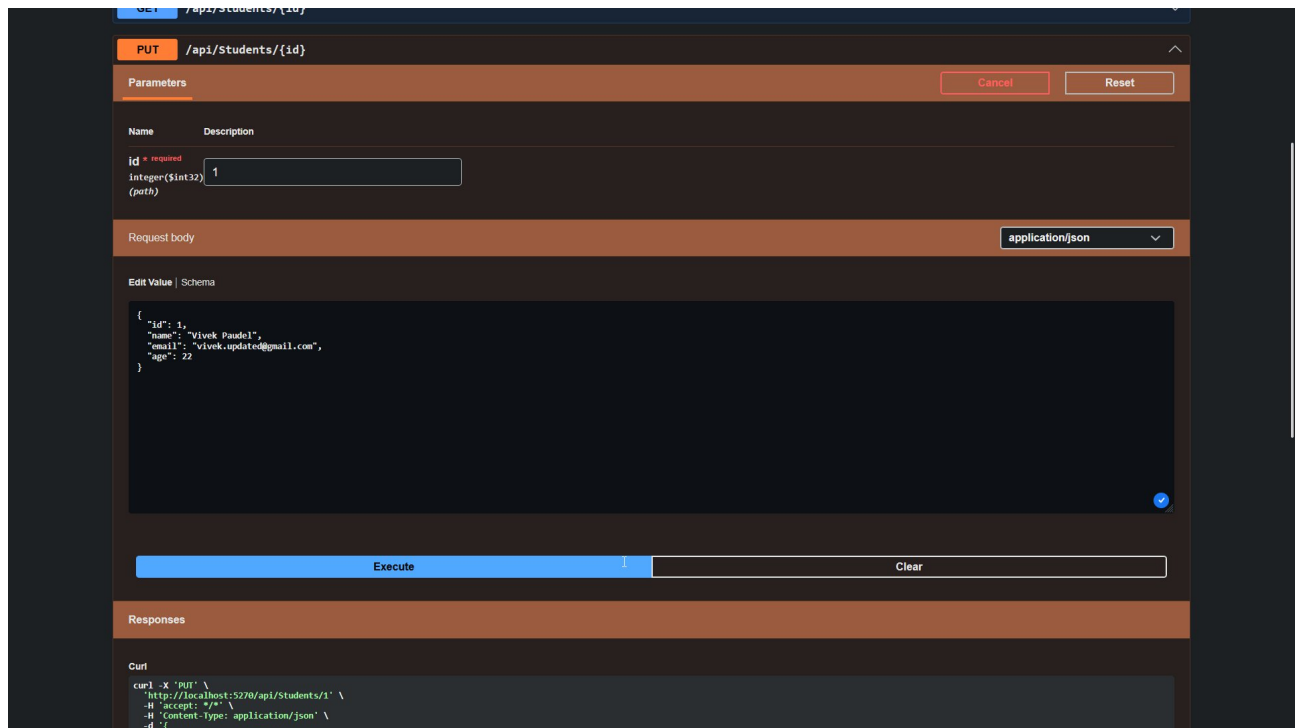


Figure 5: Swagger PUT Student update request and response.

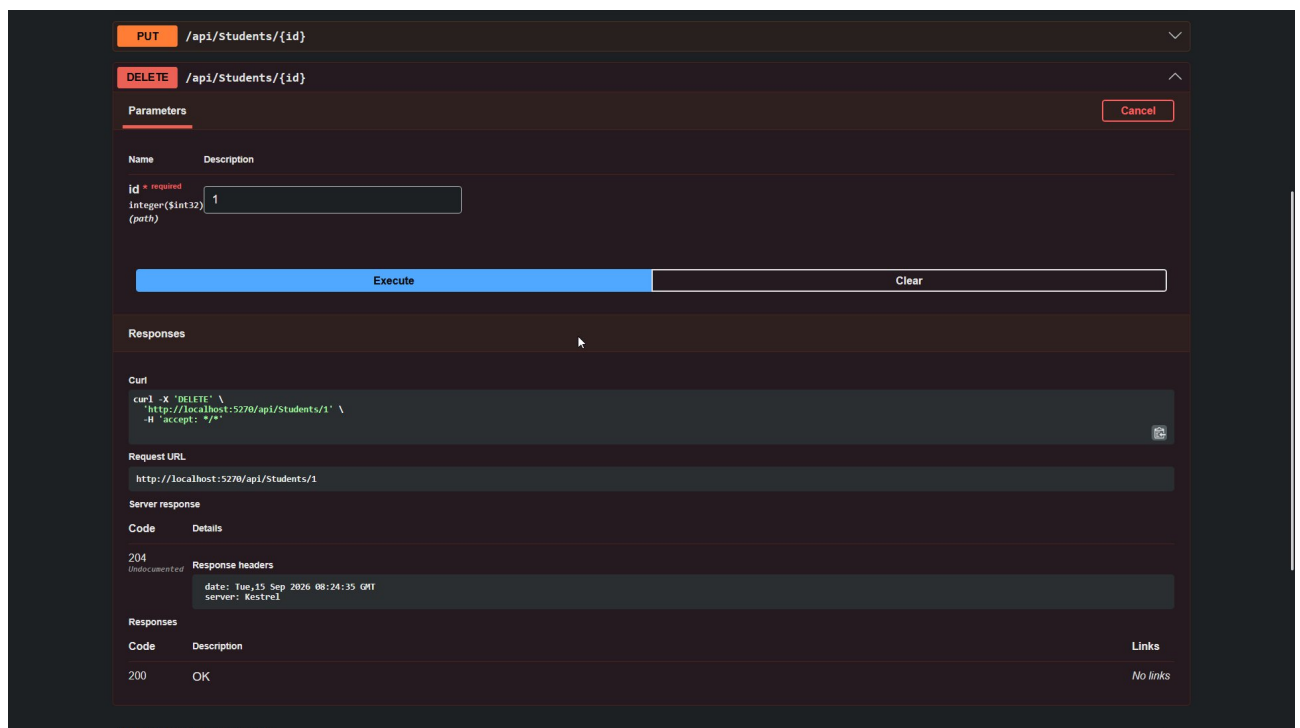


Figure 6: Swagger DELETE Student request and response.

Discussion: The Web API application was created using the ASP.NET Core Web API template, and its REST-style request/response workflow was examined using Swagger UI. The initial template endpoint illustrated interactive discovery and testing of GET operations.

An underlying SQLite database was integrated using `ApplicationDbContext` and EF Core migrations. The `StudentsController` exposes endpoints adhering to REST standard conventions: GET reads resources, POST creates records returning 201 Created, PUT updates data returning 204

No Content, and DELETE purges entities from SQLite. Swagger UI offers an interactive workbench to test HTTP payloads and status responses seamlessly.

5. Conclusion

This laboratory exercise provided practical experience with ASP.NET Core Web API and REST-based services. The WeatherForecast API and Swagger API tester demonstrated the basic request-response workflow. The exercise then connected the API to a SQLite database using Entity Framework Core, created a Student model and DbContext, and applied migrations. Finally, scaffolding/manual setup was used to create Student CRUD APIs, demonstrating how controllers expose database operations through standard HTTP methods and how Swagger can be used to test each endpoint.